

Blocco 10 - Query SQL per dashboard

Da ticket raccolti a indicatori Metabase

Relational Databases & SQL

30 min teoria + 90 min pratica

1. Dal database alla dashboard

Una dashboard non nasce da grafici: nasce da domande, granularità e query SQL difendibili.

Obiettivo del blocco

Problema

Abbiamo un DBMS con ticket raccolti e normalizzati. Ora dobbiamo costruire le query che alimentano una dashboard Metabase: KPI, trend, distribuzioni, ranking e tabelle di dettaglio.

Idea guida

Ogni card della dashboard è una domanda SQL con una granularità precisa.

Dal requisito alla card

Domanda	Card possibile
Quanti ticket sono aperti?	KPI numerico
Il backlog cresce?	Line chart giornaliera
Quali priorità pesano di più?	Bar chart per priorità e stato
Dove violiamo SLA?	Tabella o heatmap per regione e priorità
Quali categorie dominano?	Ranking categorie

Prima del grafico

- ▶ definire la popolazione: tutti i ticket o solo quelli aperti?
- ▶ definire il tempo: data apertura, data chiusura o data evento?
- ▶ definire la metrica: conteggio, tempo medio, percentuale, ranking;
- ▶ definire la granularità: giorno, priorità, regione, categoria;
- ▶ scrivere la query in modo leggibile e riutilizzabile.

Perché Metabase non elimina SQL

- ▶ l'interfaccia aiuta a costruire grafici;
- ▶ ma la semantica resta nella query;
- ▶ SQL permette di esplicitare filtri e metriche;
- ▶ le query possono essere versionate come materiale didattico;
- ▶ ogni studente può riprodurre la stessa dashboard.

2. Query KPI

Partiamo da indicatori piccoli, leggibili e verificabili.

- ▶ totale ticket ricevuti;
- ▶ ticket ancora aperti;
- ▶ ticket chiusi o risolti;
- ▶ ticket con SLA violato;
- ▶ rapporto tra violazioni SLA e totale.

Query KPI con filtri aggregati

```
SET search_path TO ticketing;

SELECT count(*) AS total_tickets,
       count(*) FILTER (WHERE closed_at IS NULL) AS open_tickets,
       count(*) FILTER (WHERE closed_at IS NOT NULL) AS closed_tickets,
       count(*) FILTER (WHERE sla_breached) AS sla_breached_tickets,
       round(
         100.0 * count(*) FILTER (WHERE sla_breached) / count(*),
         2
       ) AS sla_breach_pct
FROM dashboard_ticket_base;
```

Perché usare FILTER

- ▶ evita molte query separate;
- ▶ rende chiaro il denominatore comune;
- ▶ mantiene la metrica in una sola riga;
- ▶ è adatto a card KPI;
- ▶ è più leggibile di molte espressioni CASE WHEN annidate.

- ▶ il totale deve coincidere con il numero di righe della vista base;
- ▶ aperti più chiusi deve coincidere con il totale;
- ▶ le violazioni SLA non possono superare il totale;
- ▶ se il denominatore è zero bisogna gestire il caso;
- ▶ il significato di “aperto” deve essere dichiarato.

3. Serie temporali

Una dashboard operativa deve mostrare come il sistema cambia nel tempo.

Requisito

Mostrare per ogni giorno quanti ticket sono stati aperti, quanti sono stati risolti e se il backlog cresce.

La granularità è una riga per giorno. La data di riferimento cambia: apertura per gli ingressi, chiusura per le risoluzioni.

Query serie giornaliera

```
SELECT day,  
       opened_tickets,  
       resolved_tickets,  
       backlog_delta  
FROM dashboard_daily_flow  
ORDER BY day;
```

Che grafico scegliere

Risultato SQL	Visualizzazione
day, opened_tickets	linea o barre temporali
day, resolved_tickets	linea sovrapposta
day, backlog_delta	linea cumulata

Media mobile a 3 giorni

```
SELECT day,  
       opened_tickets,  
       round(avg(opened_tickets) OVER (  
         ORDER BY day  
         ROWS BETWEEN 2 PRECEDING AND CURRENT ROW  
       ), 2) AS opened_3d_avg  
FROM dashboard_daily_flow  
ORDER BY day;
```

Perché la media mobile è utile

- ▶ riduce rumore giornaliero;
- ▶ mantiene una riga per giorno;
- ▶ mostra trend senza nascondere il dato originale;
- ▶ usa una window function invece di una seconda tabella;
- ▶ richiede ordinamento temporale corretto.

4. Ranking e segmentazione

Le dashboard servono anche a stabilire priorità operative.

Requisito

Capire quali categorie generano più ticket e quali regioni hanno tempi medi di risoluzione più alti.

Ranking categorie

```
SELECT category,  
       count(*) AS tickets,  
       rank() OVER (ORDER BY count(*) DESC, category) AS category_rank  
FROM dashboard_ticket_base  
GROUP BY category  
ORDER BY category_rank, category;
```

- ▶ prima si costruisce una riga per categoria;
- ▶ poi la window function ordina le categorie aggregate;
- ▶ il ranking non cambia la granularità;
- ▶ un tie-breaker rende stabile l'ordinamento;
- ▶ la query diventa una card bar chart o tabella.

Tempo medio per priorità e regione

```
SELECT priority,  
       region,  
       count(*) FILTER (WHERE closed_at IS NOT NULL) AS resolved_tickets,  
       round(avg(resolution_hours), 2) AS avg_resolution_hours  
FROM dashboard_ticket_base  
WHERE closed_at IS NOT NULL  
GROUP BY priority, region  
ORDER BY priority, avg_resolution_hours DESC NULLS LAST;
```

- ▶ il filtro esclude ticket non ancora chiusi;
- ▶ il conteggio mostra quanto è robusta la media;
- ▶ una media su pochi casi va discussa;
- ▶ Metabase può usare questa query come tabella o heatmap;
- ▶ la query separa metrica e visualizzazione.

5. Filtri Metabase

Una dashboard interattiva deve filtrare senza cambiare il significato delle metriche.

- ▶ intervallo date su `opened_at`;
- ▶ priorità;
- ▶ regione;
- ▶ categoria;
- ▶ canale o sorgente.

- ▶ filtrare su data apertura non è uguale a filtrare su data chiusura;
- ▶ filtrare ticket chiusi cambia il denominatore degli SLA;
- ▶ una card KPI e una serie temporale possono avere popolazioni diverse;
- ▶ ogni filtro deve essere spiegabile;
- ▶ i nomi delle colonne nella vista aiutano Metabase a proporre filtri leggibili.

Tabella di dettaglio per drill-down

```
SELECT ticket_id,  
       source_code,  
       external_ticket_id,  
       opened_at,  
       priority,  
       category,  
       region,  
       customer_segment,  
       channel,  
       subject,  
       sla_due_at,  
       sla_breached  
FROM dashboard_ticket_base  
WHERE closed_at IS NULL  
ORDER BY priority DESC, opened_at;
```

6. Attività pratica

Costruire le query che diventeranno card Metabase.

Consegna

La direzione vuole una dashboard operativa sui ticket. Prima di aprire Metabase, bisogna scrivere e validare le query SQL che alimenteranno le card.

- ▶ input: schema `ticketing` e viste `dashboard_*`;
- ▶ output: almeno cinque query-card;
- ▶ vincolo: ogni query deve dichiarare la granularità;
- ▶ vincolo: includere KPI, trend, distribuzione, ranking e dettaglio;
- ▶ vincolo: ogni query deve essere copiabile in Metabase.

- ① eseguire `sql/ticket_architecture_dashboard_queries.sql`;
- ② scegliere quali risultati diventano KPI, grafici o tabelle;
- ③ discutere denominatori e filtri;
- ④ salvare le query in Metabase come domande;
- ⑤ assemblare le domande in una dashboard.

Organizzazione dei 90 minuti

- ▶ 15 min lettura requisiti dashboard;
- ▶ 25 min KPI e serie temporali;
- ▶ 20 min ranking e distribuzioni;
- ▶ 20 min tabella di dettaglio e filtri;
- ▶ 10 min revisione delle query prima di Metabase.

- ▶ una card Metabase è prima di tutto una query con una granularità;
- ▶ le window function aiutano trend e ranking senza perdere dettaglio;
- ▶ i filtri della dashboard devono preservare il significato della metrica.